

GROUP WORK PROJECT # 3

MScFE 632: Machine Learning in

Finance

GROUP NUMBER: 7797

FULL LEGAL NAME	LOCATION (COUNTRY)	EMAIL ADDRESS	MARK X FOR ANY NON-CONTRIBUTING MEMBER
Rishi Ram Subedi	Nepal	rishiram.subedi@gmail.com	
Vikas Gautam	Bangalore, India	vikasgtm@gmail.com	

Statement of integrity: By typing the names of all group members in the text boxes below, you confirm that the assignment submitted is original work produced by the group (excluding any non-contributing members identified with an "X" above).

Team member 1	Rishi Ram Subedi
Team member 2	Vikas Gautam
Team member 3	

Use the box below to explain any attempts to reach out to a non-contributing member. Type (N/A) if all members contributed.

Note: You may be required to provide proof of your outreach to non-contributing members upon request.

N/A

Step 2

- **Issue 1: Optimizing Hyperparameters**
 - Technical: Basic Concepts, Key Hyperparameters, Tuning Methodologies
 - Non-technical: Business Impact, Implementation Considerations, Risk Management, Best Practices for Organizations, Future Considerations

 - **Issue 3: Applying Ensemble Learning—Bagging, Boosting, or Stacking**
 - Technical: definition, formulas, steps and equations
 - Non-technical: advantages, disadvantages, use cases
- < pls refer jupyter notebook for outputs >
- Conclusion

Step 3

Issue 1: HyperParameter Tuning

3.1.1 Technical

The hyperparameters of machine learning models should be carefully tuned to get the optimal performance. This technical section will explain the basics, methods, and best practices for hyperparameter optimisation.

Basic Concepts

Hyperparameter choice corresponds to the choice of configuration variables controlling the learning process of a machine learning model. In contrast to model parameters, which are learned during training, their optimal set is to be pre-specified before actually commencing the training procedure. These variables have a large bearing on model performance, training time, and capacity for generalization.

Key Hyperparameters

Many various machine learning algorithms these days have numerous hyperparameters that one could tune:

In Neural Networks:

Learning rate: How much to update the model's weights while training

batch size: how many training examples the model sees in one iteration

number of hidden layers/neurons: The architecture of the model defines the capacity

dropout rate: how much regularization should be used to avoid overfitting

activation functions: what kind of information flows in the network

Tuning Methodologies

Several ways are there to perform systematic hyperparameter optimization:

Grid Search

Works systematically through various combinations of values for the parameters; computationally expensive but exhaustive; will surely find the best combination in the space we've provided for it. Works best when we have computation resources and not very many hyperparameters.

Random Search

Samples of random combinations of parameters. In high-dimensional spaces, much quicker than grid search. Finds good solutions with less iterations. Suffers less from the curse of dimensionality.

Bayesian Optimization

Probabilistic models to guide the search process; learn from previous evaluations to make better parameter suggestions. More efficient than both grid and random search. Particularly useful for expensive-to-evaluate objective functions.

Validation Strategies

Proper validation is crucial when performing hyperparameter tuning:

Cross-validation: Robust evaluation of parameter combinations;

Hold-out validation: Unbiased final performance estimates;

Time series cross-validation: Special considerations for temporal data;

Stratified sampling: Class distribution preserved in imbalanced datasets

One vs One or One vs All approach for validation

3.1.2 Non-Technical

Understanding hyperparameter tuning from a business and practical perspective is key to successful machine learning implementation. When organizations implement machine learning solutions, the process of hyperparameter tuning directly influences both operational

efficiency and business outcomes. This section explores the non-technical aspects that stakeholders should consider when implementing and managing hyperparameter tuning processes.

Business Impact

The impact on the business due to hyperparameter tuning stretches beyond technical aspects: a balance between computing costs and resource allocation during model training and tuning must be in place. That means not only computational expenses but also the time investment required from data science teams. Quite significant demands are to be met, infrastructure-wise, and organizations have to make the right strategic decisions toward scaling their computing resources to run hyperparameter tuning effectively.

This makes optimization so essential since the performance of models impacts business-critical KPIs directly. Well-tuned models could mean more dependable and consistent predictions, corresponding better to the desired business outcomes. Indeed, the pace with which any model deploys or its versions are updated ensures competitive advantages for an organization. Put differently, organizations that can tune their models with utmost efficiency and perform rapid deployments often do so much better in respect of their ML initiatives.

Implementation Considerations

In fact, hyperparameter tuning actually requires great forethought around team composition and skills. Every organization should make sure its teams are rightly equipped to deal with tuning exercises. Mostly, that comes from constant training and development in our people. Full documentation of knowledge-sharing practices concerns consistency in behaviours that will facilitate technical and business teams to work seamlessly together.

Another key concern in the implementation is process management. Organisations should integrate hyperparameter tuning into their workflows with ease: setting up professional version control systems and effective mechanisms of experiment tracking. Quality assurance procedures should be instituted to validate different parameter configurations' results. Besides, an organisation shall develop clear protocols for maintaining and updating its models over time.

Risk Management

Risk management in hyperparameter tuning is a wide range of activities related to technical and business challenges. Technically, organizations need to avoid overfitting when models perform well on the training data but fail in generalizing to new situations. Resources and costs must be carefully monitored to avoid overruns in budgets. Model stability and reliability

should be maintained during tuning, and technical debt burden must be kept in view by the organization.

Best Practices for Organizations

In other words, successful hyperparameter tuning involves effective strategy and execution. First, objectives and success metrics for machine learning projects should be clearly set by an organization through proper resourcing, realistic timelines, and well-defined roles and responsibilities across teams. Regular review cycles ensure that the tuning process is aligned with business objectives.

Process optimization is a significant constituent of efficient hyperparameter tuning. Organizations benefit from implementing automated tuning pipelines that can systematically explore parameter spaces. Full documentation helps maintain consistency and allows knowledge transfer between team members. Regular review and improvement cycles ensure the tuning process evolves with the organization's needs and capabilities.

Future Considerations

Organizations should also be prepared to handle the evolving needs in hyperparameter tuning. As data volumes continue to grow larger and models become more complex, so would the demands on tuning processes. This may require scaling up the infrastructure for which organizations should plan in due course of time. The landscape of tuning methodologies keeps evolving, with new tools and frameworks emerging from time to time. It is staying abreast of the industry trends and best practices that keep organizations competitive.

Innovation in hyperparameter tuning keeps on improving now and then. Finally, organizations should be aware of the latest methodologies and analyze associated benefits. New tools and frameworks may provide better efficiency or higher-quality results. Knowledge of industry trends and best practices will support an organization to make informed decisions on its tuning strategies. Besides, knowledge of the competitive landscape ensures that organizations maintain technological edge for respective markets.

By paying careful attention to these nontechnical aspects, an organization can create effective, sustainable approaches to hyperparameter tuning in support of its broader machine-learning objectives, while sustaining operational efficiency and competitive advantage.

For Stars dataset: Classification problem

Model	HyperParameters
-------	-----------------

LDA	2 principal components
SVM	{'C': 3, 'gamma': 0.001, 'kernel': 'linear'}
GridSearchCV on Neural Networks	Memory used: 2.34 MB GridSearchCV completed in 226.07 seconds. ({'activation': 'relu', 'alpha': 0.0001, 'hidden_layer_sizes': (50,) 'learning_rate': 'constant', 'solver': 'adam'}, Accuracy: 0.988)
RandomizedSearchCV on Neural Networks	Memory used: 0.21 MB RandomizedSearchCV completed in 27.39 seconds. Neural Network - Best Parameters: {'activation': 'tanh', 'alpha': 0.0101, 'hidden_layer_sizes': (100, 50), 'learning_rate': 'constant', 'solver': 'adam'} Neural Network - Best Cross-Validation Accuracy: 0.988 Neural Network - Test Accuracy: 0.983
Naive Bayes	Best Parameters: {'var_smoothing': 0.0005061} Best Cross-Validation Accuracy: 0.977

GridSearchCV on RandomForest	Memory used: 1 MB GridSearchCV completed in 138.78 seconds. Best Parameters: {'max_depth': 2, 'max_features': 'sqrt', 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 20}
RandomSearchCV on RandomForest	Memory used: 0.00 MB RandomizedSearchCV completed in 13.55 seconds. Best Parameters: {'n_estimators': 20, 'min_samples_split': 2, 'min_samples_leaf': 4, 'max_features': 'log2', 'max_depth': 2}

Regression: Neural Networks

Dataset: Tesla from yahoo finance

Optimizer Hyper Parameters:

Best Configuration:

{'lstm_units_1': 64, 'lstm_units_2': 32, 'lstm_units_3': 16, 'dense_units': 8, 'dropout_1': 0.2, 'dropout_2': 0.2, 'dropout_3': 0.2, 'learning_rate': 0.002, 'epochs': 50, 'batch_size': 32}

Test Loss: 0.000749

Issue 3:

3.3.1 Technical

Bagging (Bootstrap Aggregating):

Bagging is an ensemble learning technique that improves the stability and accuracy of machine learning models by reducing variance. It works by training multiple instances of the same model on different subsets of the data and aggregating their outputs (e.g., by averaging for regression or voting for classification).

Steps

1. Bootstrap Sampling:
 - Create multiple subsets (samples) of the training data by sampling with replacement.
 - Each subset has the same size as the original dataset but may contain duplicates due to replacement.
2. Model Training:
 - Train a base learner (e.g., decision tree, linear regression) on each subset independently.
3. Aggregation:
 - Combine the predictions from all base learners to make the final prediction:
 - Regression: Average predictions from all models.
 - Classification: Use majority voting.

Let the dataset consist of n data points: $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$

Bootstrap Sampling: Create m datasets $D_1, D_2, \dots, D_m$ where each D_i is generated by sampling n points with replacement from D .

1. Train Base Learners: Train m base models $f_1, f_2, \dots$ independently on $D_1, D_2, \dots, D_m$.
2. Aggregate Predictions:
 - Regression: $\hat{y} = 1/m \sum_{i=1}^m f_i(x)$
 - Classification: $\hat{y} = \text{mode}(\{f_1(x), f_2(x), \dots, f_m(x)\})$

Applications of Bagging

1. Random Forests:
 - Random forests are a special case of bagging where decision trees are used as base learners, and feature subsets are selected randomly at each split.
2. Regression and Classification:
 - Bagging is applied to various regression and classification tasks using models prone to overfitting.
3. Reducing Model Sensitivity:

- Effective in situations where small changes in the data can lead to large variations in model predictions.

Key Issues in Bagging

1. Bias-Variance Tradeoff:
 - Bagging primarily reduces variance but does not address model bias. If the base model has high bias, bagging cannot improve performance significantly.
2. Data Availability:
 - Requires sufficient data for effective sampling. With small datasets, bootstrap samples may not capture the full data distribution.
3. Complexity in Interpretation:
 - The aggregated model may be difficult to interpret compared to a single base learner.

Boosting

Boosting is an ensemble learning technique designed to convert weak learners (models with slightly better-than-random performance) into a strong learner. Unlike bagging, boosting sequentially trains base models, with each model focusing on correcting the errors made by its predecessor.

Steps Involved

1. Initialize Weights:
 - Each data point is assigned an initial weight to indicate its importance in training the model.
2. Iterative Training:
 - A base learner is trained on the weighted dataset.
 - The errors of the base learner are used to update the weights of the data points.
 - More weight is given to misclassified points to emphasize their importance in the next iteration.
3. Weighted Aggregation:

- Predictions from all the base models are combined with weights proportional to their accuracy.

Let the dataset be $D=\{(x_1,y_1),(x_2,y_2),\dots,(x_n,y_n)\}$

Initialize Weights: Assign initial weights $w_i(1)=1/n$ for $i=1,2,\dots,n$

1. Train Base Learners: For $t=1$ to T (number of iterations):

- Train a weak learner $h_t(x)$ on the weighted dataset.

Compute the error of the model: $\epsilon_t = \sum_{i=1}^n w_i(t) 1[y_i \neq h_t(x_i)] / \sum_{i=1}^n w_i(t)$ where $1[\cdot]$ is an indicator function.

2. Model Weight:

Compute the weight of the model h_t

$$\alpha_t = \ln(1 - \epsilon_t) / \epsilon_t$$

3. Update Data Weights:

- Update the weight for each data point:

$$w_i(t+1) = w_i(t) \exp(\alpha_t 1[y_i \neq h_t(x_i)])$$

Normalize the weights so they sum to 1.

4. Final Prediction:

- For regression: Take the weighted average: $\hat{y} = \sum_{t=1}^T \alpha_t h_t(x)$
- For classification: Take the sign of the weighted sum: $\hat{y} = \text{sign}(\sum_{t=1}^T \alpha_t h_t(x))$

Boosting Algorithms

1. AdaBoost:

- Focuses on re-weighting misclassified samples.
- Ideal for binary classification.

2. Gradient Boosting:

- Models the prediction errors (residuals) by minimizing a differentiable loss function.
- Widely used for regression and classification tasks.

3. XGBoost:

- An optimized implementation of gradient boosting with additional features like regularization, parallel processing, and handling of missing values.

Stacking

Stacking (Stacked Generalization) is an ensemble learning technique that combines predictions from multiple base models using a meta-model. Unlike bagging and boosting, stacking focuses on leveraging the complementary strengths of diverse models to improve predictive performance.

Steps Involved

1. Base Models:
 - Train multiple diverse base models (e.g., decision trees, logistic regression, support vector machines) on the same dataset.
2. Meta-Model:
 - Use the predictions from the base models as inputs for a higher-level meta-model (e.g., linear regression, neural network).
 - The meta-model learns how to combine these predictions optimally.
3. Final Prediction:
 - The meta-model generates the final predictions by aggregating the outputs of the base models.

Let $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$ be the training dataset.

1. Base Model Training: Train M base models $f_1, f_2, \dots$, on D .
2. Meta-Feature Generation:
 - For a new input x , collect predictions from all base models: $z = [f_1(x), f_2(x), \dots, f_M(x)]$ where z represents the meta-features.
3. Meta-Model Training:
 - Train a meta-model g using the meta-feature set Z and the true labels y .
4. Final Prediction:

- For a new input x , compute: $\hat{y} = g(f_1(x), f_2(x), \dots, f_M(x))$

Comparison: Bagging vs Boosting vs Stacking

Aspect	Bagging	Boosting	Stacking
Purpose	Reduce variance.	Reduce bias and variance.	Combine strengths of diverse models.
Base Models	Same type (e.g., decision trees).	Same type (e.g., weak learners).	Can be different types of models.
Aggregation	Averaging or voting.	Weighted aggregation.	Meta-model learns optimal combination.
Training	Parallel.	Sequential.	Parallel for base models, sequential for meta-model.
Performance	Robust to overfitting.	Sensitive to outliers.	Often achieves higher accuracy.
Complexity	Lower.	Medium.	Higher (requires meta-model training).

3.3.2 Non-Technical

Before we implement our model on the dataset and check our accuracy, we should consider the choice of selection of models. The model selection can be explored by trying various models (options available for supervised or unsupervised) or based on domain specifics or archival or past historical records. The model implementation is not only quantitative but also qualitative in nature.

When we try to make use of the potential of the model in terms of accuracy or other metrics. It may happen that the model suffers from over fitting or under fitting. The model may perform

well on training dataset but not so well on test dataset (bias-variance tradeoff) or may perform poorly on both the datasets (weak learners). The model may cram the entire dataset even the noise patterns while training and may not perform well on the unseen data..

When the model does not perform well upto the mark as per requirements or expectations. We may need to tweak the hyper parameters to optimum level other than the default values. If these steps does not work then we may go for Ensemble methods. These methods are based on not just one model but multiple models. This will improve the model performance and generalization.

Bagging

Advantages of Bagging

1. Variance Reduction:
 - By aggregating multiple models, bagging reduces the variance of the final prediction, leading to more stable and robust models.
2. Handles Overfitting for High-Variance Models:
 - Particularly useful for models like decision trees that tend to overfit.
3. Parallelizable:
 - Each model can be trained independently, making it easy to distribute the computation.
4. Improved Generalization:
 - Bagging helps models generalize better by averaging out noise in the predictions.

Disadvantages of Bagging

1. Increased Computational Cost:
 - Training multiple models requires more time and computational resources.
2. Ineffective for Low-Variance Models:
 - Models like linear regression and support vector machines (SVMs) are inherently low-variance, so bagging offers limited benefits.
3. Not Always Ideal for Imbalanced Data:

- Aggregation methods (e.g., voting) may not perform well if the base models are biased towards majority classes.

Boosting

Advantages of Boosting

1. High Accuracy:
 - Converts weak learners into a strong learner, achieving high predictive performance.
2. Reduces Bias and Variance:
 - Handles both underfitting (reducing bias) and overfitting (reducing variance) to some extent.
3. Customizable Loss Function:
 - Gradient Boosting allows the use of custom loss functions tailored to specific problems.
4. Feature Importance:
 - Provides insights into the significance of each feature.

Disadvantages of Boosting

1. Sensitivity to Noisy Data:
 - Boosting can overfit if there are outliers or noisy labels because it emphasizes misclassified points.
2. Sequential Nature:
 - Boosting is harder to parallelize compared to bagging, leading to higher computation times.
3. Complexity:
 - Requires careful tuning of hyperparameters (e.g., learning rate, number of estimators).
4. Risk of Overfitting:

- Particularly with too many iterations or if base learners are too complex.
 -

Stacking

Advantages of Stacking

1. **Leverages Model Diversity:**
Combines strengths of different algorithms to improve generalization.
2. **Reduces Overfitting:**
The meta-model helps correct individual model biases.
3. **Flexible Architecture:**
Allows combining models of different types (e.g., decision trees, SVMs, neural networks).
4. **Improved Performance:**
Often outperforms individual models or simpler ensemble techniques like bagging.

Disadvantages of Stacking

1. **Complexity:**
Requires careful design of base models and the meta-model.
2. **Training Time:**
Training multiple models and a meta-model increases computational cost.
3. **Risk of Overfitting:**
The meta-model may overfit if not regularized properly.
4. **Interpretability:**
Difficult to interpret the contribution of each base model.

Predicting Adult income dataset: $\leq 50K$ or $> 50k$ (classification problem)

Individual Models:

Logistic Regression Accuracy: 80.48%

Naive Bayes Accuracy: 79.53%

Decision Tree Classifier Accuracy: 80.92%

Ensemble Models: resulted in improved performance

Bagging Classifier Accuracy: 85.66%

Boosting Accuracy: 87.09%

Accuracy of Stacking Model: 85.44%

Step 4

	Wrote	Reviewed
A: Vikas	Issue 3	Issue1
B: Rishi	Issue 1	Issue 3

Step 5

Unlocking Investment Potential with Advanced ML Tools

Machine Learning is a Game-Changer not only for Investment, Medical, finance but also in other areas:

In the ever-evolving world of investments, data-driven insights are critical for staying ahead and making decision. By leveraging machine learning (ML) models, our approach transforms complicated financial data into actionable strategies, providing us with a significant edge in the market.

- **Data-Driven Decisions:** Our ML-driven insights minimize guesswork, providing us with a clear roadmap for action.
- **Customized Strategies and Solutions:** Models are tailored to our specific goals, whether it's growth, income, or risk management.
- **Real-Time Insights:** Stay agile with predictions that adapt to evolving market conditions.
- **Enhanced Efficiency:** Automating complex processes and reducing manual errors.

1. Linear Discriminant Analysis (LDA): Simplifying Complex Decisions

Value: LDA helps in classifying and forecasting investment opportunities by identifying the most relevant features that drive asset performance.

Use Cases:

- Optimal portfolio allocation by predicting market movements and sector performance trends.
- Healthcare: Diagnosing diseases by analyzing medical imaging data.
- Retail: Customer segmentation for targeted marketing campaigns.

2. Support Vector Machines (SVM): Precision-Driven Insights

Value: SVM excels in identifying patterns in non-linear data, making it ideal for analyzing volatile and noisy markets.

Use Cases:

- Pinpointing high-risk/high-reward opportunities through advanced market segmentation.
- Healthcare: Identifying cancerous cells in histopathological images.
- Cybersecurity: Detecting fraud and anomalies in network traffic.

3. Random Forest: Reliable Risk Assessment

Value: This ensemble method combines multiple decision trees to deliver robust predictions, reducing overfitting.

Use Cases:

- Risk profiling and scenario analysis for long-term investment planning.
- Healthcare: Predicting patient outcomes based on historical data.
- Agriculture: Forecasting crop yields based on weather and soil data

4. Neural Networks: Harnessing Deep Learning Power

Value: Neural networks excel at uncovering hidden trends and relationships within large datasets, enabling predictive analytics.

Use Cases:

- Forecasting stock prices and identifying market anomalies before they occur.

- Healthcare: Analyzing genetic data for personalized medicine.
- Entertainment: Recommending movies or music based on user preferences.

5. Decision Trees: Transparency and Simplicity

Value: Decision trees provide interpretable results, making it easy to understand the rationale behind predictions.

Use Cases:

- Evaluating the impact of macroeconomic factors on asset classes.
- Education: Assessing student performance and identifying intervention needs.
- Supply Chain: Optimizing inventory management decisions.

6. Bagging: Boosting Stability

Value: Bagging (Bootstrap Aggregating) enhances the stability and accuracy of models by reducing variance.

Use Cases:

- Building diversified portfolios that minimize risk while maximizing returns.
- Finance: Credit risk assessment for lending decisions.
- Energy: Predicting energy consumption patterns.

7. Boosting: Turning Weak Models into Strong Predictors

Value: Boosting techniques like AdaBoost and Gradient Boosting iteratively refine predictions, ensuring high accuracy.

Use Cases:

- Fine-tuning investment strategies to outperform benchmarks consistently.
- Retail: Dynamic pricing strategies based on consumer behavior.
- Logistics: Optimizing delivery routes for cost and time efficiency.

8. Stacking: Combining the Best of All Worlds

Value: Stacking integrates predictions from multiple models, leveraging their strengths to deliver superior results.

Use Case:

- Developing holistic strategies that outperform single-model approaches in dynamic markets.
- Healthcare: Integrating diagnostic models for comprehensive predictions.
- Finance: Creating robust trading algorithms that adapt to market shifts.

Conclusion

This suite of ML tools ensures we remain at the forefront of innovation, empowering us to make informed, strategic decisions in a competitive market. With these models, we're not just investing in assets—we're investing in cutting-edge technology that redefines success.

Across domains like healthcare, retail, education, and more, our suite of ML tools delivers unmatched value. By harnessing these technologies, we'll redefine outcomes and can achieve excellence in our field. These tools can provide a breakthrough in the trends and future of industry with machine learning!